Name - Pranav Paralkar
PRN - 202301040202
Div - C
Course / Subject - Deep Learning
Date - 16th April, 2026

Assignment 5

LSTM-Based Sequence Prediction System

1. Objective

Content:

- To build an LSTM-based model for sequence prediction
- To deploy the model using FastAPI
- To create a complete AI pipeline from training to deployment

---

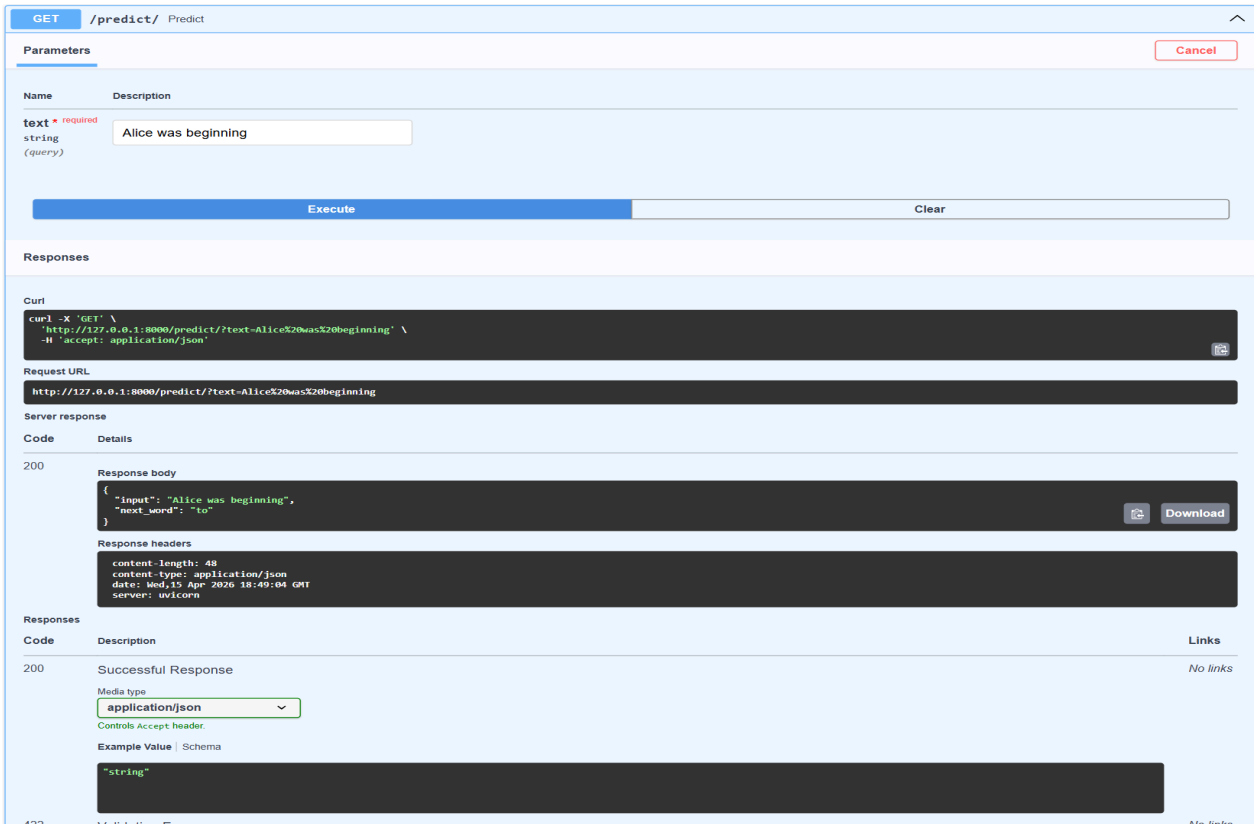
2. Problem Statement

Content:
The goal of this project is to predict the next word in a given sequence of text using an LSTM model.

Example:

Input: "alice was beginning"

Output: "to"

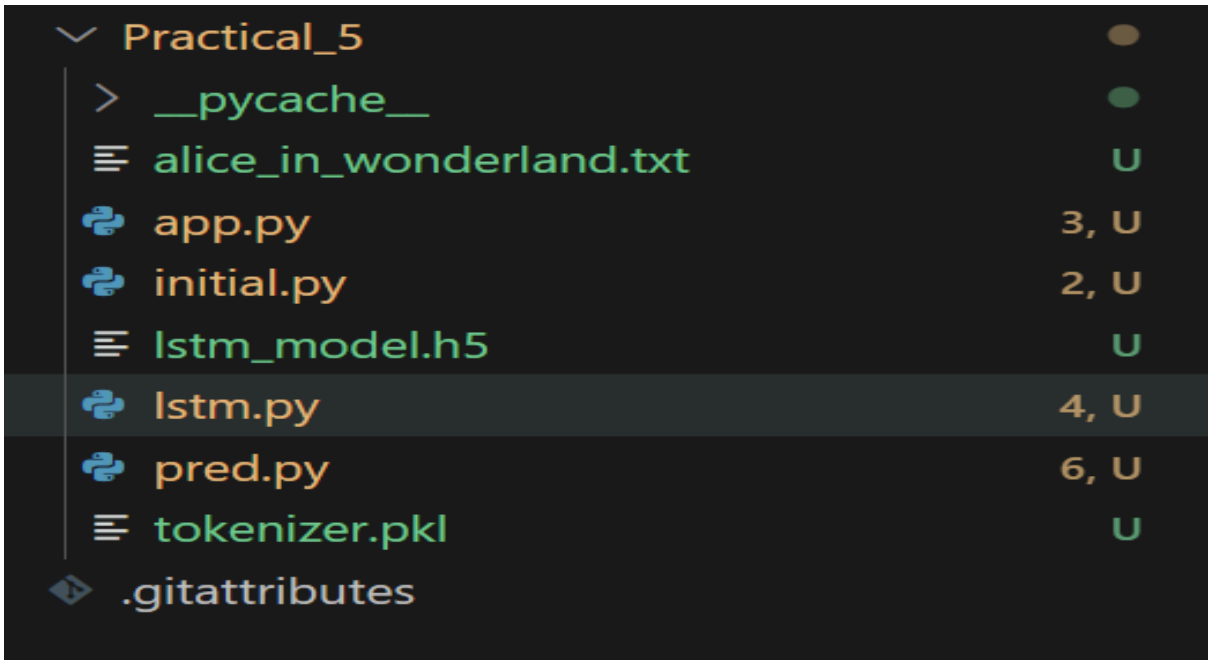


---

3. Tools & Technologies Used

Content:

- Python 3.10
- TensorFlow / Keras
- FastAPI
- Uvicorn
- VS Code

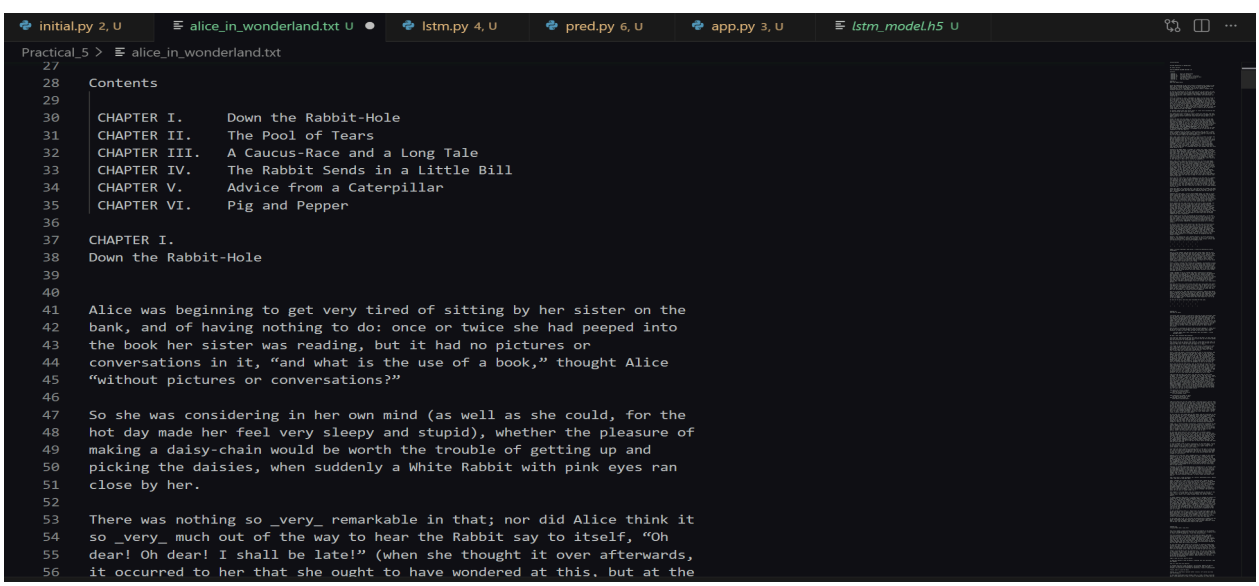


---

4. Dataset Collection

Content:

- Dataset used: *Alice in Wonderland text file*
- Source: Public domain text



5. Data Preprocessing

Content:

- Converted text to lowercase
- Tokenized words using tokenizer
- Created sequences of words
- Applied padding to make sequences equal length

Explanation:
Preprocessing converts raw text into numerical format so that the model can understand it.

initial.py 2, U | alice_in_wonderland.txt U | tokenizer.pkl U | lstm.py 4, U | pred.py 6, U | app.py 3, U

Practical_5 > initial.py > ...

```python
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load data
text = open("alice_in_wonderland.txt", "r", encoding="utf-8").read().lower()

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])

total_words = len(tokenizer.word_index) + 1

# Create sequences
input_sequences = []
for line in text.split("\n"):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_seq = token_list[:i+1]
        input_sequences.append(n_gram_seq)

# Padding
max_seq_len = max([len(x) for x in input_sequences])
input_sequences = np.array(pad_sequences(input_sequences, maxlen=max_seq_len, padding='pre'))

# Split X and y
X = input_sequences[:, :-1]
y = input_sequences[:, -1]
```

6. LSTM Model Development

Content:

- Embedding Layer: Converts words into vectors
- LSTM Layer: Learns sequence patterns
- Dense Layer: Predicts next word

initial.py 2, U | alice_in_wonderland.txt U | tokenizer.pkl U | lstm.py 4, U | pred.py 6, U | app.py 3, U

Practical_5 > lstm.py > ...

```python
import numpy as np
import pickle
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Load dataset
text = open("alice_in_wonderland.txt", encoding="utf-8").read().lower()

# Tokenize
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])

total_words = len(tokenizer.word_index) + 1

# Create sequences
input_sequences = []
for line in text.split("\n"):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[:i+1])

# Padding
max_seq_len = max(len(x) for x in input_sequences)
input_sequences = np.array(pad_sequences(input_sequences, maxlen=max_seq_len, padding='pre'))

# Split
X = input_sequences[:, :-1]
y = input_sequences[:, -1]

# Model
model = Sequential()
model.add(Embedding(total_words, 100, input_length=max_seq_len-1))
model.add(LSTM(150))
model.add(Dense(total_words, activation='softmax'))
```

7. Model Training

Content:

- Epochs: 5
- Loss function: Sparse categorical crossentropy
- Optimizer: Adam

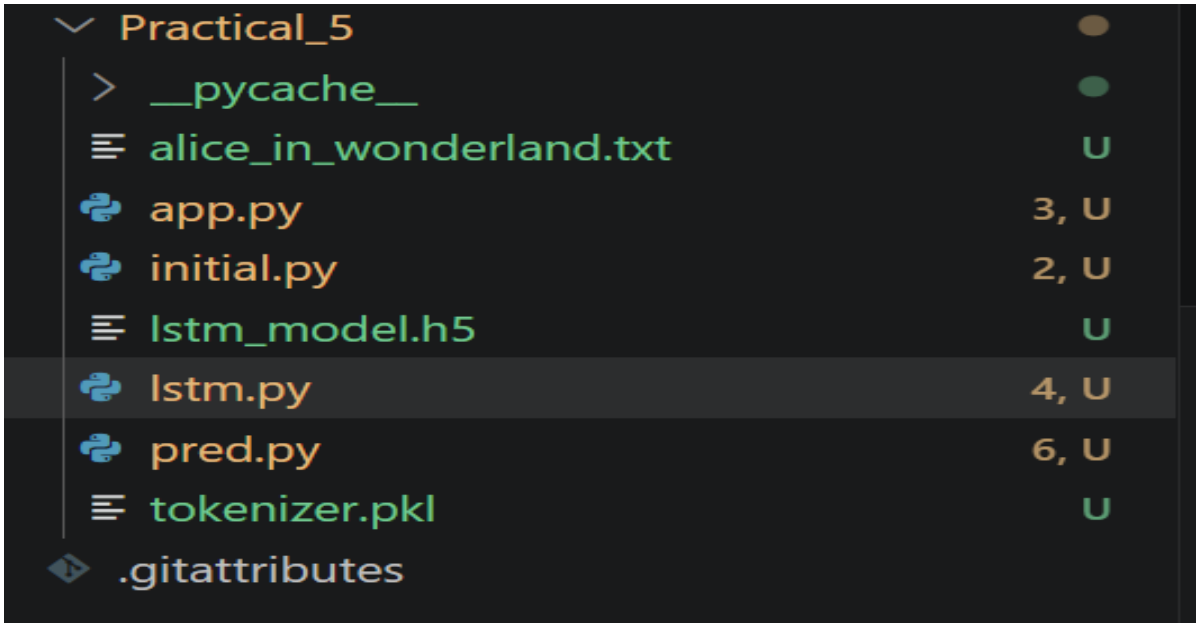
```
PS C:\Projects\DeepLearning_Assignments\Practical_5> python lstm.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776279300.376566   22468 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to float
ing-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776279302.288608   22468 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to float
ing-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
C:\Users\Pranav\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument `input_len
gth` is deprecated. Just remove it.
  warnings.warn(
I0000 00:00:1776279302.824738   22468 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performa
nce-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate comp
iler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will n
ot be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
880/880 ━━━━━━━━━━━━━━━━━━━━ 9s 9ms/step - accuracy: 0.0648 - loss: 6.2887
Epoch 2/5
880/880 ━━━━━━━━━━━━━━━━━━━━ 10s 9ms/step - accuracy: 0.0930 - loss: 5.7061
Epoch 3/5
880/880 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.1289 - loss: 5.2864
Epoch 4/5
880/880 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.1602 - loss: 4.9141
Epoch 5/5
880/880 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.1829 - loss: 4.6068
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered leg
acy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')
`.
✅ Training complete. Model saved.
PS C:\Projects\DeepLearning_Assignments\Practical_5>
```

---

8. Model Saving

Content:

- Model saved as: `lstm_model.h5`
- Tokenizer saved as: `tokenizer.pkl`



---

9. FastAPI Deployment

Content:

- Created API using FastAPI
- Endpoint: `/predict`
- Input: text
- Output: predicted next word

initial.py 2, U | alice_in_wonderland.txt U | tokenizer.pkl U | lstm.py 4, U | pred.py 6, U | app.py 3, U

Practical_5 > app.py > predict

```python
from fastapi import FastAPI
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.sequence import pad_sequences
import pickle

app = FastAPI()

model = load_model("lstm_model.h5")
tokenizer = pickle.load(open("tokenizer.pkl", "rb"))
# Use model input shape so API padding always matches training.
max_seq_len = int(model.input_shape[1]) + 1

@app.get("/")
def home():
    return {"message": "LSTM Text Predictor API"}

@app.get("/predict/")
def predict(text: str):
    token_list = tokenizer.texts_to_sequences([text])[0]
    token_list = pad_sequences([token_list], maxlen=max_seq_len-1, padding='pre')

    predicted = model.predict(token_list)
    predicted_index = int(np.argmax(predicted, axis=-1)[0])  # FIX

    output_word = ""

    for word, index in tokenizer.word_index.items():
        if index == predicted_index:
            output_word = word
            break

    return {"input": text, "next_word": output_word}
```

10. Running the API

Content:
Command used:

uvicorn app:app --reload

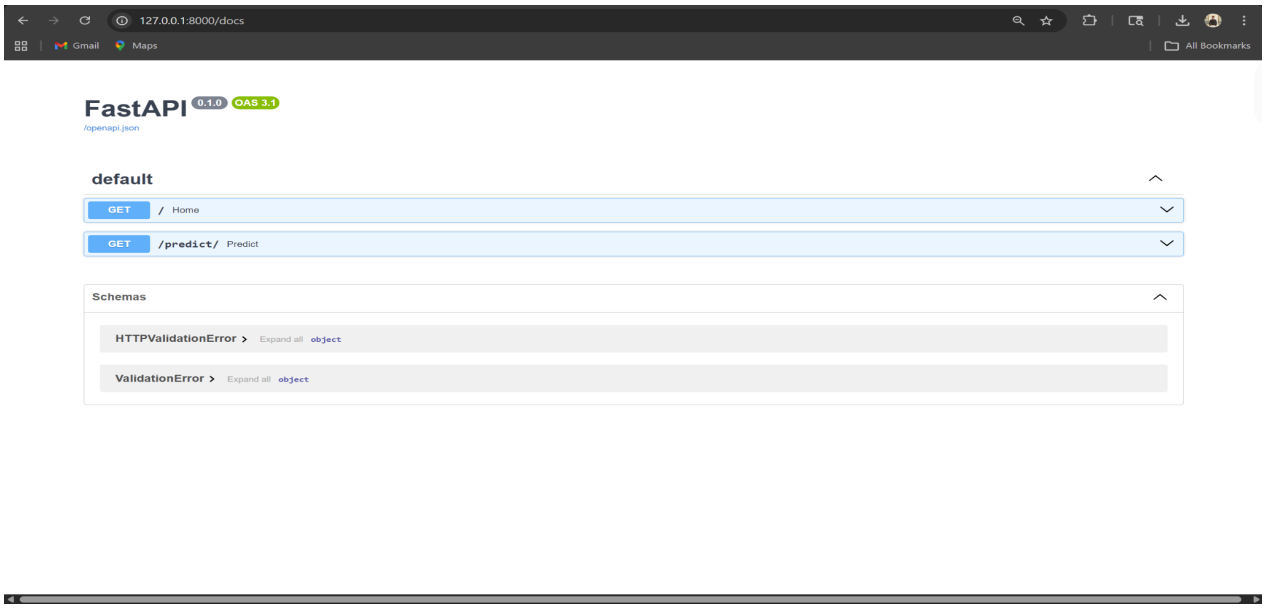
```
PS C:\Projects\DeepLearning_Assignments\Practical_5> uvicorn app:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Projects\\DeepLearning_Assignments\\Practical_5']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [10952] using StatReload
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776279469.789334   10184 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical resu
lts due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_
ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776279471.782831   10184 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical resu
lts due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_
ENABLE_ONEDNN_OPTS=0`.
I0000 00:00:1776279472.168782   10184 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instruct
ions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the
appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are insta
lled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty unti
l you train or evaluate the model.
INFO:     Started server process [4280]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

11. API Testing (Swagger UI)

Content:

- Accessed API at:

http://127.0.0.1:8000/docs



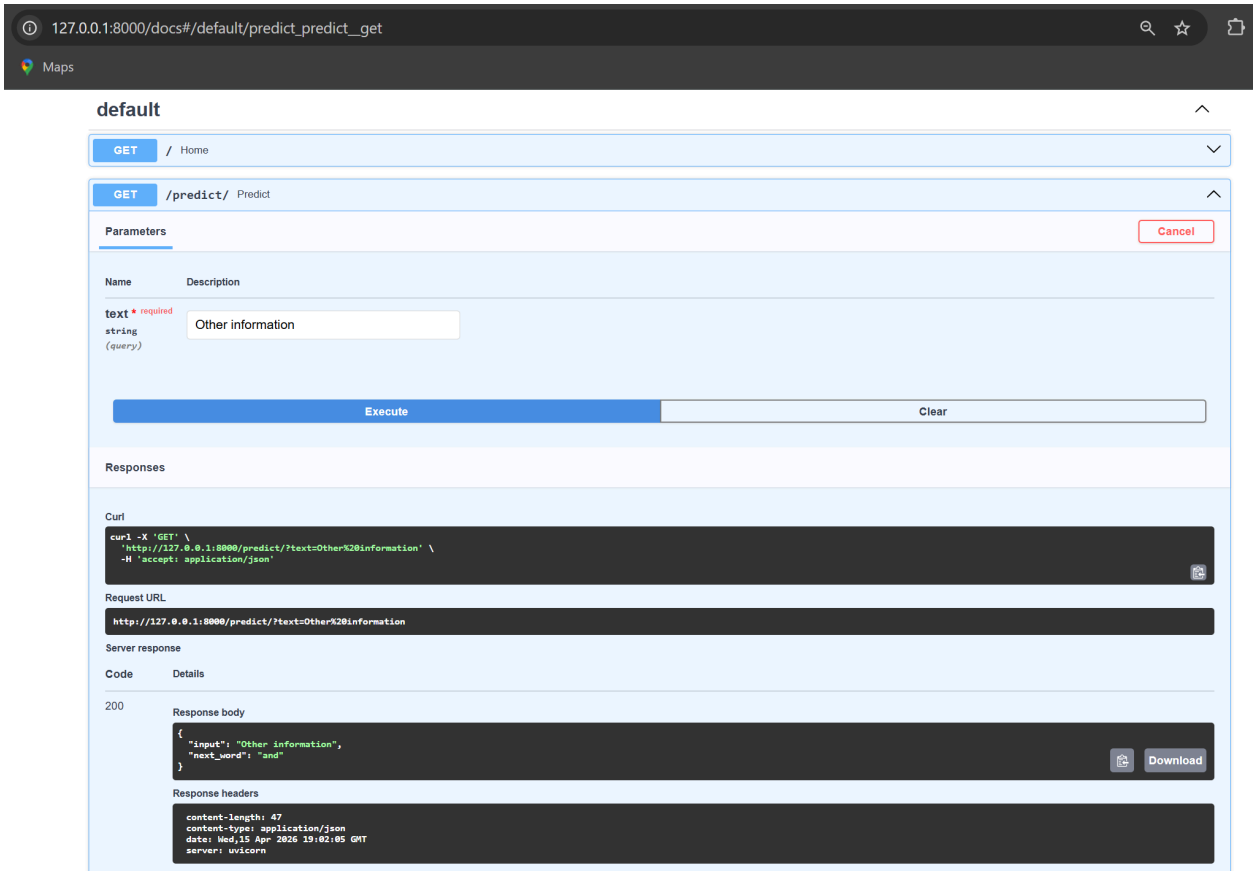
12. Results

Content:

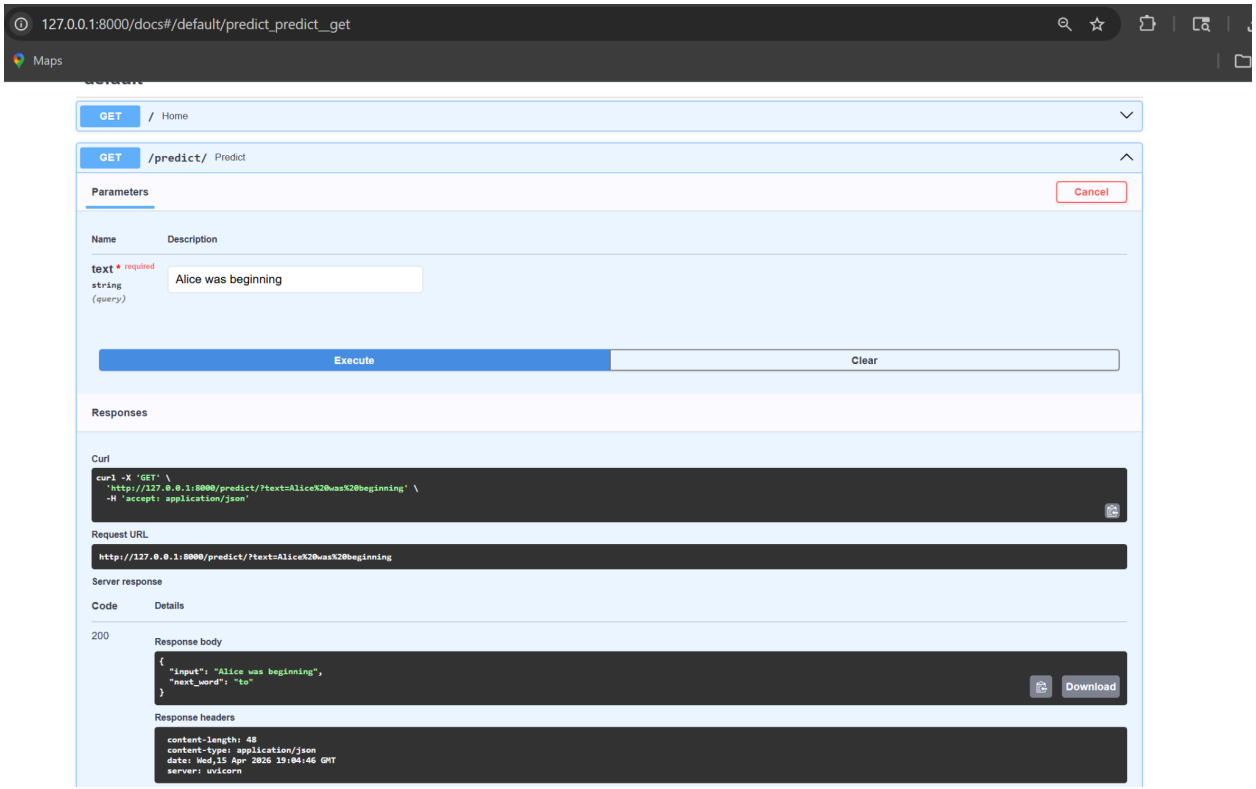
- Model successfully predicts next word
- Example outputs generated

Successful Predictions:

```
Release date: June 27, 2008 [eBook #11]
                Most recently updated: June 26, 2025

Language: English

Other information and formats: www.gutenberg.org/ebooks/11

Credits: Arthur DiBianca and David Widger


*** START OF THE PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***

[Illustration]


Alice’s Adventures in Wonderland
```



Alice was beginning to get very tired of sitting by her sister on the bank, and of having nothing to do: once or twice she had peeped into the book her sister was reading, but it had no pictures or conversations in it, “and what is the use of a book,” thought Alice “without pictures or conversations?”



13. LSTM Mathematical Model

Components of LSTM

1. Forget Gate

The forget gate determines which information from the previous cell state should be discarded.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- f_t: Forget gate output
- h_{t-1}: Previous hidden state
- x_t: Current input
- σ: Sigmoid function

This gate outputs values between 0 and 1.

- 0 → completely forget
- 1 → completely retain

2. Input Gate

The input gate decides what new information should be added to the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

3. Candidate Cell State

This creates new candidate values that could be added to the cell state.

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

4. Cell State Update

The cell state is updated by combining previous memory and new information.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- Old information is partially forgotten
- New relevant information is added

This is the **core memory mechanism** of LSTM.

5. Output Gate

The output gate determines what part of the cell state is used to produce the output.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

6. Hidden State (Final Output)

The hidden state is the final output of the LSTM at a given time step.

$$h_t = o_t \cdot \tanh(C_t)$$

---

14. Conclusion

This project successfully demonstrates the development of an end-to-end **LSTM-based sequence prediction system** for next word prediction. The model was trained on textual data to learn contextual relationships between words using the memory capabilities of LSTM networks. Through preprocessing techniques such as tokenization, sequence generation, and padding, raw text data was effectively converted into a format suitable for training.

The trained model was able to generate meaningful next-word predictions, indicating its ability to capture sequential dependencies in language. Furthermore, the deployment of the model using FastAPI transformed it into a functional and accessible API, enabling real-time predictions through a web interface.

During the development process, several practical challenges such as dependency issues, model errors, and API integration were encountered and resolved, contributing to a deeper understanding of both deep learning workflows and deployment strategies.

Overall, this project not only reinforces theoretical concepts of LSTM and sequence learning but also provides hands-on experience in building and deploying industry-relevant AI systems.

---

15. AI Tool Usage

Content:

Tool Used: ChatGPT (OpenAI)

Purpose:

- Code generation
- Debugging
- Concept explanation

Sections Used:

- Model development
- API deployment
- Error fixing

16. References (Use for Implementation & Learning)

API Development

- FastAPI Official Documentation
  https://fastapi.tiangolo.com/
- FastAPI Youtube Referenc Link-
  https://www.youtube.com/watch?v=WJKsPchji0Q&list=PLKnIA16_RmvZ41tjbKB2ZnwchfniNsMuQ

LSTM & Deep Learning

- TensorFlow/Keras Documentation
  https://www.tensorflow.org/api_docs

- Sequence Models Tutorials (LSTM)
  https://keras.io/examples/nlp/text_generation/